

M3 CLI Tool User Guide

M3CLI-01

February 2026

Version 1.0

Table of Contents

- Preface
- 1. Setup Guide
 - Install
 - Update & Healthcheck
 - Uninstall
- 2. Usage Guide
 - Configuration
 - Usage Examples
 - Common Use Cases
- 3. Support
 - How to Report an Issue
 - How to Communicate with the Support Team

We're here to help you every step of the way. If you encounter any issues or have any questions, don't hesitate to reach out to our [support team](#).

Preface

About This Guide

Maestro CLI, also known as M3-CLI, is a robust command-line interface (CLI) tool designed to provide a powerful alternative to user interfaces (UI). It allows you to perform cloud infrastructure management operations, as well as manage related assets, by sending respective calls to Maestro cloud management platform. The guide is aimed at Maestro users who plan to manage their virtual infrastructure using the CLI interface. Here, you'll find all the resources you need to install, update, uninstall, and effectively use the Maestro CLI.

Target Audience

The guide is designed for Maestro users who aim to reach Maestro functionality via a CLI interface.

Related Documents

- For the detailed information on the CLI commands, their parameters and usage, please refer to M3-CLI Reference Guide.
- For the information on Maestro platform administering, to M3 Admin User Guide.

M3-CLI Setup Guide

Tool Installation

Pre-requisites

Before you start with the installation of M3-CLI, ensure you have **Python 3.10+**, **pip** and **venv** installed on your system. If not, follow the guides provided below:

- [Python3 Installation Guide for Windows](#)
- [Python3 Installation Guide for Linux](#)
- [Python3 Installation Guide for Mac](#)

Installation Flow

Tip: It's recommended to use a virtual environment to protect against dependency conflicts. You can install the `m3cli` tool without this step, but if you install it in a virtual environment, you will need to activate that environment before using the tool.

Create a Virtual Environment on Windows

 [Guide for virtualenv installation](#)

1. Create the virtual environment:

```
virtualenv -p python3 venv
```

2. Activate the virtual environment:

```
venv\Scripts\activate.bat
```

Create a Virtual Environment on Linux/Mac

 [Guide for virtualenv installation](#)

1. Create the virtual environment:

```
virtualenv -p python3 .venv
```

2. Activate the virtual environment:

```
source .venv/bin/activate
```

Install M3-CLI

3. To install the M3-CLI tool, use the following command:

```
pip install m3-cli
```

Tool Update

Pre-Requisites

The tool update requires the same pre-requisites as installation.

CLI Update Detection Flow

Once a day, when any CLI command is invoked, the CLI performs a check for available updates. This includes making the `m3 health-check` call to the server and using its response to refresh the data stored in the `default.cr` file about the latest CLI version and distribution links.

This data is used between daily update checks to notify users of pending updates while avoiding the need to communicate with the server on each request.

An update notification is printed to the user before the actual response and contains instructions on how to update the CLI client on Linux, Windows, or macOS machines.

If the `m3 health-check` call fails due to a server-side error while the CLI performs a daily update check, a warning is printed informing the user of the error and that the CLI is unable to automatically check for updates.

To force the CLI to check for an update, invoke the `m3 health-check` command manually.

Update Instructions

Windows

1. Check your current version:

```
pip list
```

2. Install the update, using the latest CLI version as `m3-cli` parameter, for example:

```
pip install m3-cli==3.150.0
```

Example:

```
(venv) C:\Maestro3\m3-cli>pip list
Package            Version
-----
attrs              23.2.0
...
m3-cli             3.101.10
...
(venv) C:\Maestro3\m3-cli>pip install m3-cli==3.150.0
...
Installing collected packages: m3-cli
  Attempting uninstall: m3-cli
    Found existing installation: m3-cli 3.101.10
    Uninstalling m3-cli-3.101.10:
      Successfully uninstalled m3-cli-3.101.10
  Successfully installed m3-cli-3.150.0
```

3. Verify the installation:

```
m3 --help
m3 health-check
```

Linux/Mac**1. Activate your virtual environment (if using one):**

```
source .venv/bin/activate
```

2. Check your current version:

```
pip list
```

3. Install the update, using the latest CLI version as m3-cli parameter, for example:

```
pip install m3-cli==3.150.0
```

Example:

```
(.venv) user@localhost:~/maestro3/m3-cli$ pip list
```

```
Package            Version
-----
attrs              23.2.0
...
m3-cli             3.101.10
...
```

```
(.venv) user@localhost:~/maestro3/m3-cli$ pip install m3-cli==3.150.0
```

```
Collecting m3-cli==3.150.0
```

```
  Downloading m3_cli-3.150.0-py3-none-any.whl (245 kB)
    245.8/245.8 kB 2.1 MB/s eta 0:00:00
```

```
Installing collected packages: m3-cli
```

```
  Attempting uninstall: m3-cli
```

```
    Found existing installation: m3-cli 3.101.10
```

```
    Uninstalling m3-cli-3.101.10:
```

```
      Successfully uninstalled m3-cli-3.101.10
```

```
Successfully installed m3-cli-3.150.0
```

4. Verify the installation:

```
m3 --help
m3 health-check
```

Verification Examples

If the M3-CLI version is outdated, the health-check command will return:

```
(venv) C:\Maestro3\m3-cli>m3 health-check
```

```
=====
You are using an outdated version of m3-cli (3.101.10). Version 3.150.0 is now
available.
Please use the next link to download cli for Linux -
https://mcc.cloud.epam.com/cli/3.101.10/m3-cli-x86_64.AppImage
To upgrade cli on macOS run the following command: pip install
https://mcc.cloud.epam.com/cli/3.101.10/m3-cli-3.101.10.zip
For Windows run: pip install https://mcc.cloud.epam.com/cli/3.101.10/m3-cli-
3.101.10.zip
=====
```

Response:

```
+-----+-----+-----+-----+
| Server Available | Mails | Ownership | Terraform | Cli Latest Version |
+=====+=====+=====+=====+=====+
```

True	True	True	True	3.150.0	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+

If the M3-CLI version is up to date:

```
$ m3 health-check
```

Response:

+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
Server Available	Mails	Ownership	Terraform	Cli Latest Version	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
True	True	True	True	3.150.0	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+

Uninstallation

Pre-Requisites

The tool uninstallation requires the same pre-requisites as installation.

Uninstallation Flow

Windows

1. To uninstall the tool, run the following command:

```
pip uninstall m3-cli
```

Example:

```
(venv) C:\Maestro3_tests\m3-cli-test-updating>pip uninstall m3-cli
Found existing installation: m3-cli 3.150.0
Uninstalling m3-cli-3.150.0:
  Would remove:
    c:\maestro3_tests\m3-cli-test-updating\venv\lib\site-packages\m3_cli-3.150.0.dist-info\*
    c:\maestro3_tests\m3-cli-test-updating\venv\lib\site-packages\m3cli\*
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\access_meta.json
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\bash_m3cli_complete.sh
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\commands_def.json
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\m3cli_complete.py
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\zsh_m3cli_complete.sh
    c:\maestro3_tests\m3-cli-test-updating\venv\scripts\m3.exe
Proceed (Y/n)? y
Successfully uninstalled m3-cli-3.150.0
```

Linux/Mac

1. Activate your virtual environment (if using one):

```
source .venv/bin/activate
```

2. To uninstall the tool, run the following command:

```
pip uninstall m3-cli
```

Example:

```
(.venv) user@localhost:~/maestro3_tests/m3-cli-test-updating$ pip uninstall m3-cli
Found existing installation: m3-cli 3.150.0
Uninstalling m3-cli-3.150.0:
  Would remove:
```

```
/home/user/maestro3_tests/m3-cli-test-updating/.venv/lib/python3.10/site-  
packages/m3_cli-3.150.0.dist-info/*  
/home/user/maestro3_tests/m3-cli-test-updating/.venv/lib/python3.10/site-  
packages/m3cli/*  
/home/user/maestro3_tests/m3-cli-test-updating/.venv/m3cli/access_meta.json  
/home/user/maestro3_tests/m3-cli-test-  
updating/.venv/m3cli/bash_m3cli_complete.sh  
/home/user/maestro3_tests/m3-cli-test-updating/.venv/m3cli/commands_def.json  
/home/user/maestro3_tests/m3-cli-test-updating/.venv/m3cli/m3cli_complete.py  
/home/user/maestro3_tests/m3-cli-test-updating/.venv/m3cli/zsh_m3cli_complete.sh  
/home/user/maestro3_tests/m3-cli-test-updating/.venv/bin/m3  
Proceed (Y/n)? y  
Successfully uninstalled m3-cli-3.150.0
```

Tip: Uninstalling M3-CLI does not delete all libraries that were installed with M3-CLI. For complete removal, delete the `venv` (Windows) or `.venv` (Linux/Mac) directory.

[Contents](#)

Usage Guide

Configuration

Before using M3-CLI, execute the `m3 access` command to configure all necessary settings.

Automatic Configuration

Run the following command and follow the interactive prompts:

```
m3 access
```

Manual Configuration

To configure settings manually, set up the following environment variables:

- `M3SDK_ACCESS_KEY`: Specifies an M3 access key associated with your Maestro3 user
- `M3SDK_SECRET_KEY`: Specifies the secret key associated with the access key (essentially the “password” for the access key)
- `M3SDK_ADDRESS`: Specifies the address of the Maestro3 environment

Non-Interactive Configuration

Alternatively, you can set credentials non-interactively using the `m3 access` command with specified parameters:

```
(venv) C:\Maestro3\m3-cli>m3 access --access_key <access_key> --secret_key  
<secret_key> --api_address <api_address>
```

After running this command, a `default.cr` file with your access parameters will be created in the `.m3cli` folder inside your home directory.

Obtaining Credentials

Obtain Maestro3 URL

Before obtaining your credentials, you need the Maestro3 base URL - the web address of your Maestro3 instance used to configure CLI/SDK connectivity.

How to obtain your Maestro3 URL:

Option 1: From Your Login Page - Use the address you access the Maestro3 web interface with - Format: `https://<your-maestro3-domain>/`

Option 2: Contact Your Administrator - Reach out to your DevOps/Platform team or System Administrator - The deployment team will have this information from the initial setup

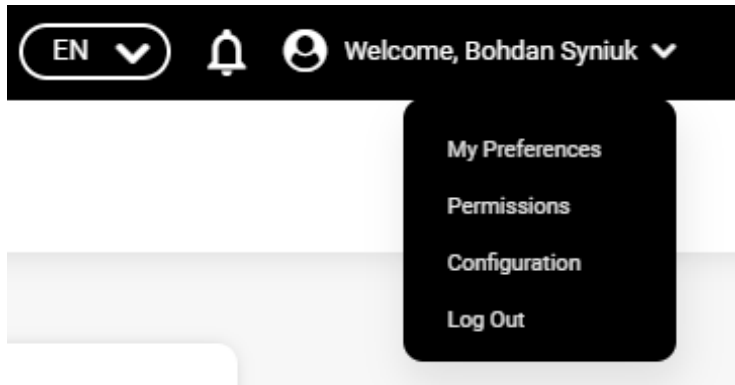
Option 3: Check Internal Documentation - Review your organization’s onboarding materials - Check internal wikis, knowledge bases, or welcome emails

Note: Once you have this URL, you’ll use it as the `--api_address` parameter in the configuration methods described above ([Automatic](#), [Manual](#), or [Non-Interactive](#)). Use only the base domain without path suffixes (e.g., remove `/login`).

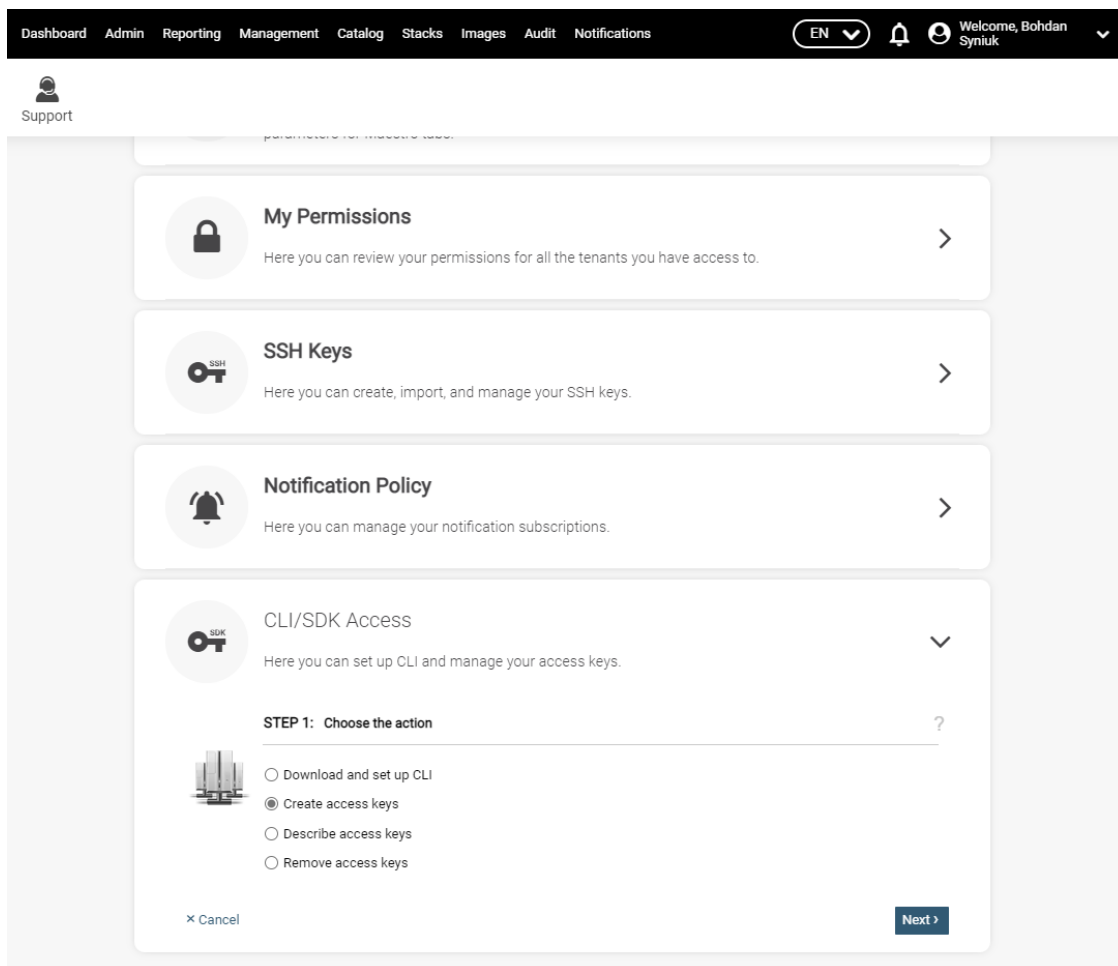
Generating Access Credentials

To obtain your credentials, log in to the Maestro3 UI and follow these steps:

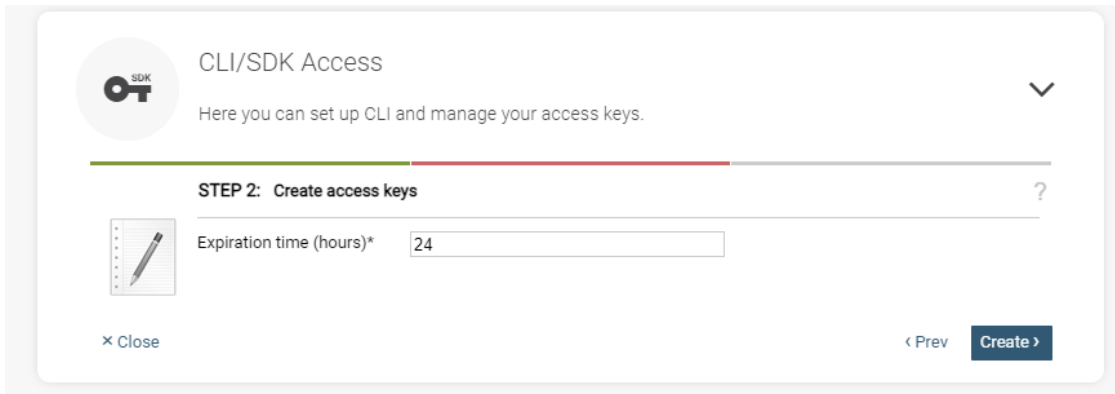
1. Switch to the My Account page:



2. Start the 'CLI/SDK Access' wizard:

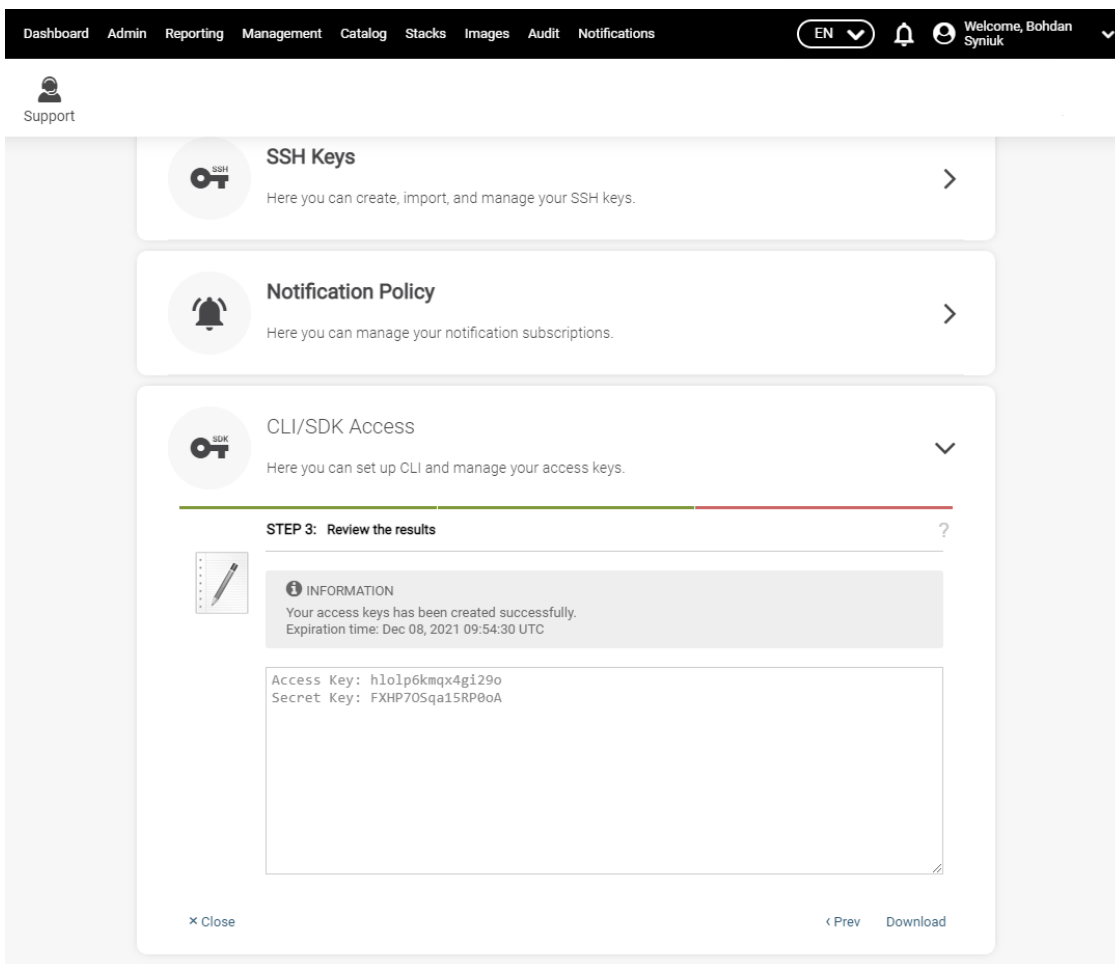


3. Specify the expiration time for the credentials:



The screenshot shows a modal window titled "CLI/SDK Access" with a sub-header "Here you can set up CLI and manage your access keys." Below this is a progress bar with "STEP 2: Create access keys" highlighted. A form field for "Expiration time (hours)*" contains the value "24". At the bottom, there are buttons for "Close", "Prev", and "Create".

4. Obtain your credentials and export the access key to environment variable M3SDK_ACCESS_KEY, and the secret key to M3SDK_SECRET_KEY:



The screenshot shows the "CLI/SDK Access" modal window at "STEP 3: Review the results". It displays an "INFORMATION" message: "Your access keys has been created successfully. Expiration time: Dec 08, 2021 09:54:30 UTC". Below this, the generated credentials are shown: "Access Key: h1o1p6kmqx4gi29o" and "Secret Key: FXHP70Sqa15RP0oA". At the bottom, there are buttons for "Close", "Prev", and "Download".

Optional Configuration

The following environment variables can be used to override the default values that M3-CLI uses:

- **M3CLI_CONFIGURATION_FOLDER_PATH:** Specifies the path to the directory containing files with command and parameter definitions (commands_def.json, commands_help.py). Default value: internal application path.
- **M3SDK_VERSION:** Specifies the SDK version to use. Default version: 3.150.0.
- **M3CLI_DEBUG:** Set to True to enable debug mode. Default value: False.
- **LOG_PATH:** Specifies a custom path to store the M3-CLI log file.

Usage Examples

After installing the tool and setting the credentials as environment variables, M3-CLI is ready to use.

Tip: The development of Maestro3 CLI is ongoing. Examples below contain demo data and will be updated.

To get information about available commands and parameters, run `m3` as displayed below:

Root Help

The root help displays data about all available commands:

```
$ m3
-----
Description:
This is the newly created M3CLI.
Usage: m3 <command> [options]

-----
| COMMANDS | ALIAS | HELP |
-----+-----+-----+
| access   | access | Set up all needed settings. You can use the command in non-interactive mode by specifying parameters: |
|          |        | --access_key, --secret_key, --api_address |
|-----+-----+-----+
| activate-platform-service | actpfs | Activate platform service |
|-----+-----+-----+
| add-schedule-instances | addschin | Adds existing schedule to instances |
|-----+-----+-----+
| create-image | cim | Creates image |
|-----+-----+-----+
| create-instance-quota | cinstq | Creates instance quota |
|-----+-----+-----+
| create-keypair | ckey | Creates keypair |
|-----+-----+-----+
| create-schedule | csch | Use this command to create schedules. |
|-----+-----+-----+
| create-volume-quota | cvolq | Creates volume quota |
|-----+-----+-----+
| deactivate-platform-service | dactpfs | Deactivate platform service |
|-----+-----+-----+
| delete-image | delim | Deletes image |
|-----+-----+-----+
| delete-instance-quota | delinstq | Removes instance quota |
|-----+-----+-----+
| delete-keypair | delkey | Deletes key |
|-----+-----+-----+
| delete-private-tag | deltag | Removes tag from the instance. This command is working only for Private Clouds! |
|-----+-----+-----+
| delete-schedule | delsch | Use this command to delete schedule. |
|-----+-----+-----+
| delete-schedule-instances | delschin | Removes existing schedule to instances |
|-----+-----+-----+
| delete-script | delscr | Removes init script |
|-----+-----+-----+

```

Command Help

The `--help` flag displays abbreviated parameter information:

```
(.venv)
MINGW64 /c/data/m3cli/examples/env
$ m3 din --help

Usage: m3 describe-instances(din) [options]

Use --full-help parameter for all available options

Options:

--region,      -r,      * The region name
--tenant-name, -tn,      * Tenant name
--instance-ids, -iids,   Instances IDs
--resource-tag, -tag,    Names of the resource tag: test1key:test1value
```

Full Command Help

Use the `--full-help` flag to get all available options:

```
MINGW64 /c/data/m3cli/examples/env
$ m3 din --full-help

-----
Description:

Use this command to return information about instances that you own.

Examples:

1. Describes all available instances for the certain tenant in the specified region

    m3 describe-images -tn <tenantName> -r <region>

2. Describes all instances for the certain tenant in region with specified instance ID

    m3 describe-images -tn <tenantName> -r <region> -ids <instanceId>

3. Describes an instances with specified IDs

    m3 describe-images -tn <tenantName> -r <region> -id <instanceId1,instanceId2>

Usage: m3 describe-instances(din) [options]

Options:

--help                Show abbreviated help message and exit.
--full-help           Show full help message and exit.
--verbose             Writes logs into "m3cli.log" file
--json               Show the server response in the JSON format
--table              Show the server response in the Table format (default)
--full               Show the server full response without special formatting (can be used with --json view type)

--region,      -r,      * The region name
--tenant-name, -tn,      * Tenant name
--instance-ids, -iids,   Instances IDs
--resource-tag, -tag,    Names of the resource tag: test1key:test1value
```

Output Modes

Table output mode (default):

```
$ m3 din --region AWS-EUCENTRAL --tenant-name MSTR- --instance-ids i-0dc9f-
Response:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| instanceId | tenant | state | shape | privateIpAddress | keyName | imageId | owner | platform |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| i-0dc9f- | AWS-MSTR- | running | Custom | 10.0. |  | Unknown |  | Linux |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

JSON output mode:

```
$ m3 din --region AWS-EUCENTRAL --tenant-name MSTR [REDACTED] --instance-ids i-0dc9f368e[REDACTED] --json
[
  {
    "imageId": "Unknown",
    "instanceId": "i-0dc9f368e[REDACTED]",
    "keyName": "[REDACTED] server",
    "platform": "Linux",
    "privateIpAddress": "10.0.[REDACTED]",
    "publicDnsName": "[REDACTED]",
    "tenant": "AWS-MSTR[REDACTED]",
    "state": "running",
    "shape": "Custom"
  }
]
```

Full output mode:

```
$ m3 din --region AWS-EUCENTRAL --tenant-name MSTR [REDACTED] --instance-ids i-0dc9f368e[REDACTED] --full
Response:

instances:
- architecture = x86_64
  availabilityZone = eu-central-1a
  cloud = AWS
  cpuCount = 1
  creationDate = Mar 24, 2021 17:03:16 UTC
  creationDateTimestamp = 1616605396000
  deviceNames:
  - /dev/sda1
  disableApiTermination = null
  diskInfos:
  - deviceName = /dev/sda1
    diskId = vol-03356918d[REDACTED]
    system = true
  imageId = Unknown
  instanceId = i-0dc9f368e[REDACTED]
  instanceName = [REDACTED] server
  keyName = [REDACTED] server
  memoryMb = null
  modificationDate = Mar 24, 2021 17:03:16 UTC
  nativeImageId = ami-07670[REDACTED]
  owner = null
  platform = Linux
  privateDnsName = ip-10-[REDACTED].eu-central-1.compute.internal
  privateIpAddress = 10.0.[REDACTED]
  publicDnsName = [REDACTED]-central-1.compute.amazonaws.com
  publicIpAddress = [REDACTED]
  region = AWS-EUCENTRAL
  regionId = [REDACTED]
  securityGroupsNames:
  - [REDACTED]
  - [REDACTED]
  - basic
  - [REDACTED]
  shape = Custom
  state = running
  subnetId = [REDACTED]
  tags:
  - key = [REDACTED]
    value = [REDACTED] server
  tenant = AWS-MSTR[REDACTED]
  tenantInRegionId = [REDACTED]
```

Tip: The `--verbose` parameter writes the command result to the terminal.

Common Use Cases

This section provides practical use cases and workflows for common tasks performed with M3-CLI. Each use case includes step-by-step instructions and examples.

Use Case List

1. [Getting Started](#)
2. [Instance Lifecycle Management](#)
3. [Storage Management](#)
4. [SSH Key Management](#)
5. [Scheduling and Automation](#)
6. [Tagging and Organization](#)
7. [Infrastructure as Code with Terraform](#)
8. [Cost Management and Reporting](#)
9. [Platform Services Management](#)
10. [Networking Configuration](#)
11. [Monitoring and Insights](#)

Getting Started Use Case

Use Case: First-Time Setup and System Check

Scenario: You've just installed M3-CLI and need to verify your setup and access to the Maestro3 platform.

Steps:

1. Configure your credentials:

```
m3 access --access_key <your_access_key> --secret_key <your_secret_key> --
api_address <api_address>
```

2. Verify system status:

```
m3 health-check
```

Expected output:

Response:

```
+-----+-----+-----+-----+-----+
| Server Available | Mails | Ownership | Terraform | Cli Latest Version |
+-----+-----+-----+-----+-----+
| True            | True  | True      | True      | 3.150.0             |
+-----+-----+-----+-----+-----+
```

3. List available tenants:

```
m3 describe-tenants --cloud AWS
```

4. List available regions for your tenant:

```
m3 describe-regions --tenant <tenant_name>
```

Instance Lifecycle Management

Use Case 1: Launching a New Instance

Scenario: You need to launch a new web server instance in AWS.

Steps:**1. List available images:**

```
m3 describe-images --region us-east-1 --tenant <tenant_name> --json
```

2. List available shapes (instance types):

```
m3 describe-shapes --cloud AWS --tenant <tenant_name> --region us-east-1
```

3. Create/Import SSH key if needed:

```
m3 create-key --key-name web-server-key --cloud AWS --region us-east-1 --tenant <tenant_name>
```

4. Launch the instance:

```
m3 run-instances \  
  --image ami-12345678 \  
  --name web-server-01 \  
  --key-name web-server-key \  
  --number-of-instances 1 \  
  --cloud AWS \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --shape t3.medium \  
  --tag "Environment:Production,Application:WebServer,Owner:TeamA"
```

5. Verify the instance is running:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --running
```

Use Case 2: Managing Instance State

Scenario: You need to stop instances during non-business hours and start them again in the morning.

Steps:**1. Stop an instance:**

```
m3 stop-instances --instance-id i-0123456789abcdef0 --tenant <tenant_name> --region us-east-1
```

2. Verify the instance is stopped:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --instance-id i-0123456789abcdef0 --stopped
```

3. Start the instance again:

```
m3 start-instances --instance-id i-0123456789abcdef0 --tenant <tenant_name> --region us-east-1
```

4. Reboot an instance if needed:

```
m3 reboot-instances --instance-id i-0123456789abcdef0 --region us-east-1 --tenant <tenant_name>
```

Use Case 3: Terminating Instances with Protection

Scenario: You need to terminate a test instance while ensuring production instances are protected.

Steps:**1. Enable termination protection on the production instance:**

```
m3 manage-termination-protection \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --instance-id i-prod123456 \  
  --action enable
```

2. Terminate the test instance:

```
m3 terminate-instances \  
  --instance-id i-test123456 \  
  --region us-east-1 \  
  --tenant <tenant_name>
```

3. Verify termination:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --instance-id i-  
test123456
```

Storage Management

Use Case 1: Adding Storage to an Existing Instance

Scenario: Your application instance is running out of disk space and needs additional storage.

Steps:**1. Check current storage:**

```
m3 describe-storages --region us-east-1 --tenant <tenant_name> --instance-id i-  
0123456789abcdef0
```

2. Create and attach new storage:

```
m3 create-attach-storage \  
  --name app-data-vol-02 \  
  --instance-id i-0123456789abcdef0 \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --size 100
```

3. Verify storage attachment:

```
m3 describe-storages --region us-east-1 --tenant <tenant_name> --instance-id i-  
0123456789abcdef0
```

Use Case 2: Moving Storage Between Instances

Scenario: You need to move a data volume from one instance to another.

Steps:**1. Detach storage from the source instance:**

```
m3 detach-storage \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --size 100
```

```
--instance-id i-source123 \  
--storage-id vol-0123456789
```

2. Wait for detachment to complete, then verify:

```
m3 describe-storages --region us-east-1 --tenant <tenant_name> --storage-id vol-  
0123456789
```

3. Attach the volume to the target instance:

Note: Use the Maestro3 UI or appropriate cloud-specific commands to attach the detached volume to the new instance.

Use Case 3: Storage Cleanup

Scenario: Remove unused storage volumes to reduce costs.

Steps:

1. List all storage volumes:

```
m3 describe-storages --region us-east-1 --tenant <tenant_name> --json
```

2. Identify unattached volumes and delete them:

```
m3 delete-storage \  
--tenant <tenant_name> \  
--region us-east-1 \  
--storage-id vol-unused123
```

SSH Key Management

Use Case: Setting Up SSH Access for a Team

Scenario: You need to set up SSH keys for team members across multiple regions.

Steps:

1. Import an existing team member's public key:

```
m3 import-key \  
--file-path /path/to/john_doe_key.pub \  
--key-name john-doe-key \  
--cloud AWS \  
--region us-east-1 \  
--tenant <tenant_name>
```

2. Create a new key for automation purposes:

```
m3 create-key \  
--key-name automation-key \  
--cloud AWS \  
--region us-east-1 \  
--tenant <tenant_name>
```

3. List all available keys:

```
m3 describe-keys --cloud AWS --region us-east-1 --tenant <tenant_name>
```

4. Replicate the key to another region:

```
m3 manage-key \  
  --cloud AWS \  
  --key-name automation-key \  
  --region us-west-2 \  
  --action add \  
  --tenant <tenant_name>
```

5. Remove unused keys:

```
m3 delete-key --key-name old-unused-key
```

Scheduling and Automation

Use Case 1: Creating Cost-Saving Schedules

Scenario: You want to automatically stop development instances at 7 PM and start them at 8 AM on weekdays.

Steps:

1. Create a stop schedule for the evening:

```
m3 create-schedule \  
  --action STOP \  
  --cloud AWS \  
  --region us-east-1 \  
  --cron "0 0 19 * * MON-FRI" \  
  --description "Stop dev instances at 7 PM weekdays" \  
  --name dev-evening-stop \  
  --schedule-type "My instances with tag" \  
  --tenant <tenant_name> \  
  --tag-key Environment \  
  --tag-value Development
```

2. Create a start schedule for the morning:

```
m3 create-schedule \  
  --action START \  
  --cloud AWS \  
  --region us-east-1 \  
  --cron "0 0 8 * * MON-FRI" \  
  --description "Start dev instances at 8 AM weekdays" \  
  --name dev-morning-start \  
  --schedule-type "My instances with tag" \  
  --tenant <tenant_name> \  
  --tag-key Environment \  
  --tag-value Development
```

3. Verify the schedules:

```
m3 describe-schedules --cloud AWS --tenant <tenant_name> --region us-east-1
```

Use Case 2: Managing Instance-Specific Schedules

Scenario: Add or remove schedules from specific instances.

Steps:

1. Add a schedule to specific instances:

```
m3 add-schedule-instances \
  --cloud AWS \
  --region us-east-1 \
  --tenant <tenant_name> \
  --instance-id i-0123456,i-0123457,i-0123458 \
  -dn dev-evening-stop
```

2. Remove a schedule from instances:

```
m3 delete-schedule-instances \
  --cloud AWS \
  --region us-east-1 \
  --instance-id i-0123456 \
  -dn dev-evening-stop \
  --tenant <tenant_name>
```

3. Delete a schedule entirely:

```
m3 delete-schedule --region us-east-1 --name dev-evening-stop --tenant <tenant_name>
```

Tagging and Organization

Use Case: Implementing a Tagging Strategy

Scenario: You need to tag instances according to your organization's tagging policy for cost allocation and management.

Steps:

1. Tag a new instance during launch:

```
m3 run-instances \
  --image ami-12345678 \
  --name app-server-01 \
  --key-name my-key \
  --number-of-instances 1 \
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --shape t3.medium \
  --tag "Project:CustomerPortal,Environment:Production,Owner:john.doe@company.com"
```

2. Add tags to existing instances:

```
m3 set-tags \
  --cloud AWS \
  --instance-id i-0123456789abcdef0 \
  --tenant <tenant_name> \
  --region us-east-1 \
  --tag "Compliance:PCI-DSS,BackupPolicy:Daily"
```

3. View all tags on an instance:

```
m3 describe-tags \
  --cloud AWS \
  --instance-id i-0123456789abcdef0 \
  --tenant <tenant_name> \
  --region us-east-1
```

4. Find all instances with a specific tag:

```
m3 describe-instances \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --resource-tag "Environment:Production"
```

5. Find untagged instances:

```
m3 describe-instances \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --group UNTAGGED
```

6. Remove specific tags:

```
m3 delete-tags \  
  --cloud AWS \  
  --instance-id i-0123456789abcdef0 \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --tag "OldTag,AnotherOldTag"
```

Infrastructure as Code with Terraform

Use Case 1: Deploying Infrastructure from Terraform Template

Scenario: You need to deploy a complete application stack using a Terraform template.

Steps:

1. Upload the Terraform template:

```
m3 upload-terraform-template \  
  --source-path /path/to/terraform/template \  
  --terraform-version v1.5.0 \  
  --cloud AWS \  
  --tenant <tenant_name> \  
  --template web-app-stack \  
  --variables "environment:production,instance_count:3,instance_type:t3.large"
```

2. Generate a variable file for review:

```
m3 generate-platform-service-varfile \  
  --cloud AWS \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --service web-app-stack
```

3. Plan the deployment (dry-run):

```
m3 plan-terraform-template \  
  --cloud AWS \  
  --template web-app-stack \  
  --tenant <tenant_name> \  
  --variables "environment:production,instance_count:3,instance_type:t3.large"
```

4. Apply the template:

```
m3 apply-terraform-template \  
  --cloud AWS \  
  --template web-app-stack
```

```
--template web-app-stack \  
--tenant <tenant_name>
```

5. Check deployment status:

```
m3 describe-terraform-stacks \  
--cloud AWS \  
--tenant <tenant_name>
```

Use Case 2: Managing Terraform Template Locks

Scenario: Prevent concurrent modifications to critical infrastructure.

Steps:

1. Lock the template before making changes:

```
m3 lock-terraform-template \  
--cloud AWS \  
--template production-infrastructure \  
--tenant <tenant_name> \  
--expiration 2
```

2. Make your changes and prolong the lock if needed:

```
m3 prolong-terraform-template-lock \  
--cloud AWS \  
--template production-infrastructure \  
--tenant <tenant_name> \  
--expiration 1
```

3. Unlock when done:

```
m3 unlock-terraform-template \  
--cloud AWS \  
--template production-infrastructure \  
--tenant <tenant_name>
```

Use Case 3: Template Cleanup

Scenario: Remove unused Terraform templates and their resources.

Steps:

1. List all templates:

```
m3 describe-terraform-templates --cloud AWS --tenant <tenant_name>
```

2. Destroy resources created by the template:

```
m3 destroy-terraform-stack \  
--cloud AWS \  
--template old-test-stack \  
--tenant <tenant_name>
```

3. Delete the template:

```
m3 delete-terraform-template \  
--cloud AWS \  
--template old-test-stack \  
--tenant <tenant_name>
```

Cost Management and Reporting

Use Case 1: Monthly Cost Analysis

Scenario: Generate comprehensive cost reports for the current month.

Steps:

1. Get total costs for the month:

```
m3 total-report \
  --tenant-group <tenant_group_name> \
  --year 2024 \
  --month 03
```

2. Get a detailed breakdown by service:

```
m3 subtotal-report \
  --tenant-group <tenant_group_name> \
  --year 2024 \
  --month 03
```

3. Get resource-level details:

```
m3 resource-report \
  --tenant-group <tenant_group_name> \
  --year 2024 \
  --month 03 \
  --day 15
```

4. Export to JSON for further analysis:

```
m3 resource-report \
  --tenant-group <tenant_group_name> \
  --year 2024 \
  --month 03 \
  --json > monthly_costs.json
```

Use Case 2: Identifying Cost Optimization Opportunities

Scenario: Find resources that are driving costs and can be optimized.

Steps:

1. Generate a cost and usage report:

```
m3 cost-usage-report \
  --cloud AWS \
  --date 15.03.2024 \
  --region us-east-1 \
  --resource-type INSTANCE \
  --tenant <tenant_name>
```

2. Find untagged resources (potential waste):

```
m3 untagged-resource-report \
  --tenant <tenant_name> \
  --from 01.03.2024 \
  --to 31.03.2024
```

3. Review insights for specific instances:

```
m3 describe-insights \  
  --instance-id i-0123456789abcdef0 \  
  --cloud AWS \  
  --region us-east-1 \  
  --tenant <tenant_name>
```

4. Check budget status:

```
m3 budgets-report \  
  --tenant-group <tenant_group_name> \  
  --year 2024 \  
  --month 03
```

Use Case 3: Multi-Tenant Cost Reporting

Scenario: Generate cost reports across multiple tenants for management review.

Steps:

1. Generate a multi-tenant report:

```
m3 multitenant-report \  
  --from 01.03.2024 \  
  --to 31.03.2024 \  
  --region us-east-1 \  
  --report-type summary \  
  --include-billing-source \  
  --json > multitenant_costs.json
```

Platform Services Management

Use Case: Publishing and Deploying a Platform Service

Scenario: You've developed a custom service and want to make it available to your tenants.

Steps:

1. Upload the service script:

```
m3 upload-script \  
  --cloud AWS \  
  --source-path /path/to/init-script.sh \  
  --script monitoring-agent-installer \  
  --tenant <tenant_name>
```

2. Publish the service:

```
m3 publish-platform-service \  
  --service monitoring-service \  
  --service-version 2.1.0 \  
  --template monitoring-template \  
  --discoverable-url https://docs.company.com/monitoring \  
  --title "Enterprise Monitoring Service" \  
  --supported-cloud AWS,AZURE,GOOGLE \  
  --cloud AWS \  
  --tenant <tenant_name> \  
  --provider-version v1.5.0 \  
  --category Monitoring,Security \  
  --description "Comprehensive monitoring solution" \  
  --icon-path /path/to/icon.png
```


3. Add usage documentation:

```
m3 add-service-section \  
  --service monitoring-service \  
  --section USAGE \  
  --block-title "Getting Started" \  
  --block-value "Install the agent on your instances using the provided script..."
```

4. Activate the service for a tenant:

```
m3 activate-platform-service \  
  --cloud AWS \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --service monitoring-service
```

5. Verify service activation:

```
m3 describe-platform-service-stacks \  
  --cloud AWS \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --service monitoring-service
```

6. Deactivate when needed:

```
m3 deactivate-platform-service \  
  --cloud AWS \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --service-id <service_id>
```

Networking Configuration

Use Case 1: Setting Up Static IPs

Scenario: Configure static IP addresses for your instances.

Steps:**1. Allocate a static IP:**

```
m3 allocate-ip \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --network vpc-0123456789
```

2. List available IPs:

```
m3 describe-ip --tenant <tenant_name> --region us-east-1
```

3. Associate the IP with an instance:

```
m3 associate-ip \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --static-ip 203.0.113.25 \  
  --instance-id i-0123456789abcdef0
```

4. Disassociate when needed:

```
m3 disassociate-ip \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --static-ip 203.0.113.25
```

5. Release unused IP:

```
m3 release-ip \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --ip 203.0.113.25
```

Use Case 2: VLAN Management

Scenario: Set up network segmentation using VLANs.

Steps:

1. List available VLANs:

```
m3 describe-vlans --tenant <tenant_name> --region us-east-1
```

2. Activate a VLAN:

```
m3 activate-vlan \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --vlan-name production-vlan-100 \  
  --description "Production environment network"
```

3. Move an instance to the VLAN:

```
m3 move-to-vlan \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --vlan-name production-vlan-100 \  
  --instance-id i-0123456789abcdef0
```

4. Deactivate VLAN when no longer needed:

```
m3 deactivate-vlan \  
  --tenant <tenant_name> \  
  --region us-east-1 \  
  --vlan-name old-dev-vlan-50
```

Monitoring and Insights

Use Case 1: Monitoring Instance Health and Performance

Scenario: Regular monitoring and insights gathering for your infrastructure.

Steps:

1. List all running instances:

```
m3 describe-instances \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --running
```

2. Get detailed insights for a specific instance:

```
m3 describe-insights \  
  --instance-id i-0123456789abcdef0 \  
  --cloud AWS \  
  --region us-east-1 \  
  --tenant <tenant_name>
```

3. Check instances with high-risk factors:

```
m3 describe-instances \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --insights \  
  --group VULNERABLE
```

4. View recent events:

```
m3 describe-events \  
  --tenant <tenant_name> \  
  --cloud AWS \  
  --number-of-events 50 \  
  --search-type RESOURCE \  
  --resource-id i-0123456789abcdef0 \  
  --region us-east-1
```

Use Case 2: Resource Inventory and Compliance

Scenario: Maintain an accurate inventory of all resources for compliance purposes.

Steps:

1. Get a complete resource inventory:

```
m3 describe-resources \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --json > resource_inventory.json
```

2. Filter by specific tags:

```
m3 describe-resources \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --tag "Compliance:SOC2"
```

3. Find untagged resources:

```
m3 describe-resources \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --group UNTAGGED
```

4. Check instances by owner:

```
m3 describe-instances \  
  --region us-east-1 \  
  --tenant <tenant_name> \  
  --owner john.doe@company.com
```

Use Case 3: Creating Custom Images

Scenario: Create reusable images from configured instances.

Steps:**1. Create an image from an instance:**

```
m3 create-image \
  --name web-server-baseline-v2 \
  --instance-id i-0123456789abcdef0 \
  --region us-east-1 \
  --tenant <tenant_name> \
  --description "Web server with Apache 2.4, PHP 8.1, and security hardening"
```

2. List available images:

```
m3 describe-images --region us-east-1 --tenant <tenant_name>
```

3. Delete an old image:

```
m3 delete-image \
  --image-id ami-old123456 \
  --region us-east-1 \
  --tenant <tenant_name>
```

Advanced Workflows**Workflow 1: Complete Application Deployment**

Scenario: Deploy a complete 3-tier web application with proper tagging, monitoring, and cost controls.

Steps:**1. Set up SSH keys:**

```
m3 create-key --key-name app-deployment-key --cloud AWS --region us-east-1 --tenant
<tenant_name>
```

2. Deploy the database tier:

```
m3 run-instances \
  --image ami-database \
  --name app-db-01 \
  --key-name app-deployment-key \
  --number-of-instances 1 \
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --shape r5.xlarge \
  --tag "Tier:Database,Application:CustomerApp,Environment:Production"
```

3. Deploy the application tier:

```
m3 run-instances \
  --image ami-app-server \
  --name app-server \
  --key-name app-deployment-key \
  --number-of-instances 3 \
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --shape c5.large \
  --tag "Tier:Application,Application:CustomerApp,Environment:Production"
```

4. Deploy the web tier:

```
m3 run-instances \
  --image ami-web-server \
  --name web-server \
  --key-name app-deployment-key \
  --number-of-instances 2 \
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --shape t3.medium \
  --tag "Tier:Web,Application:CustomerApp,Environment:Production"
```

5. Set up cost-saving schedules for non-production environments (if deploying dev/test):

```
m3 create-schedule \
  --action STOP \
  --cloud AWS \
  --region us-east-1 \
  --cron "0 0 19 * * MON-FRI" \
  --description "Stop non-prod instances at 7 PM" \
  --name non-prod-evening-stop \
  --schedule-type "My instances with tag" \
  --tenant <tenant_name> \
  --tag-key Environment \
  --tag-value Development
```

6. Monitor the deployment:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --resource-tag
"Application:CustomerApp" --running
```

Workflow 2: Disaster Recovery Exercise

Scenario: Create backups and prepare for disaster recovery.

Steps:**1. Create images of critical instances:**

```
m3 create-image \
  --name prod-db-backup-2024-03-15 \
  --instance-id i-prod-db-01 \
  --region us-east-1 \
  --tenant <tenant_name> \
  --description "Production database backup before upgrade"
```

2. Document the current state:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --json > pre-dr-
state.json
m3 describe-storages --region us-east-1 --tenant <tenant_name> --json > pre-dr-
storage.json
```

3. Set up a backup service:

```
m3 backup \
  --tenant <tenant_name> \
  --region us-east-1 \
  --cloud AWS \
  --service-id backup-svc-01 \
```

```
--instance-id i-prod-db-01 \  
--backup-server-id backup-srv-01
```

4. Verify backups and generate reports:

```
m3 describe-events \  
--tenant <tenant_name> \  
--cloud AWS \  
--number-of-events 100 \  
--search-type BACKUP \  
--region us-east-1
```

Best Practices

- **Always use JSON output for automation:**

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --json | jq  
'instances[] | select(.state=="running")'
```

- **Use tags consistently:**

- Environment (Production, Development, Testing)
- CostCenter
- Owner
- Application
- Compliance requirements

- **Implement schedules for cost savings:**

- Stop non-production instances outside business hours
- Use appropriate instance sizes

- **Perform regular monitoring:**

```
# Add to cron or scheduled task  
m3 describe-instances --region us-east-1 --tenant <tenant_name> --insights --  
json > daily-insights-$(date +%Y%m%d).json
```

- **Use variable files for Terraform templates** to maintain consistency across environments.

- **Enable termination protection** on production instances.

- **Conduct regular cost reviews:**

```
# Monthly cost review script  
m3 total-report --tenant-group <tenant_group> --year $(date +%Y) --month $(date  
+%m) --json > monthly-costs.json
```

Contents

Support

How to Report an Issue

When reporting an issue, please provide as much detail as possible, including:

1. **Python version:** The M3-CLI tool supports Python 3.10+
2. **Clear description:** A concise description of the issue
3. **Steps to reproduce:** Detailed steps to reproduce the issue
4. **Error messages:** Any error messages or logs that can help us understand the problem

Please send your issue report to: SupportSyndicateTeam@epam.com

How to Communicate with the Support Team

When communicating with the support team, please be respectful and patient. Our team is here to help you, and we will do our best to resolve your issue as quickly as possible.

Tips for Effective Communication:

1. **Be clear and concise** in your descriptions
2. **Provide all necessary information** related to your issue
3. **Include screenshots or logs** if available

Contact Information

- **Email:** SupportSyndicateTeam@epam.com
 - **Response Time:** We aim to respond to all queries within 7 calendar days (5 business days, excluding weekends)
-

Tool Information

- **Company:** EPAM
 - **Tool:** M3-CLI
 - **Supported Python Versions:** 3.10+
 - **Support Contact:** SupportSyndicateTeam@epam.com
-

[Contents](#)

Version History

Version	Date	Summary
v.1.0	February 9, 2026	Initial version finalized

View Changelog: <https://github.com/Maestro-Cloud-Control/m3-cli/blob/main/CHANGELOG.md>

The Documentation generated for: **m3, version 4.158.1**